

Lab 2 - ALU, Regfile, and Datapath Design

Raymond Tran, Robert Bulanon, Anna Larson, Dhiab Vaheedudheen

I. INTRODUCTION

This document details the ALU and regfile implementation. Included is the testing metrics and design details.

II. ALU INSTRUCTION IMPLEMENTATION

Thus far, the following instructions have been implemented: ADD, ADDI, ADDU, ADDUI, ADDC, ADDCI, SUB, SUBC, CMP, CMPI, CMPI, AND, OR, XOR, NOT, LSH, LSHI, ALSH, NOP.

The only instructions that have been postponed until the implementation of the decoder are ADDCUI and SUBI. These two instructions, without the decoder, complicate the responsibilities and implementation of the ALU. Without the correct operand-formatting rule for both operations, they cannot yet be included within the list of instructions.

III. REGFILE DESIGN

The register file holds 16 general-purpose 16-bit registers (Registers[0:15]), addressed by 4-bit fields (WriteAddress, ReadAddressA, ReadAddressB), plus a separate 5-bit StoredFlags register for the program status (C, L, F, Z, N). It has one synchronous write port and two independent, fully combinational read ports.

A. Architecture Rationale

With instructions utilizing two-operands (ADD Rd, Rs1, Rs2-style), the implementation needs to read two source registers and produce one result in the same instruction cycle, so two simultaneous, independent read ports are the natural match. One write port is enough because an instruction only ever commits one destination register per cycle. Splitting the flags out into their own register, with their own enable, reflects that flags are conceptually a different piece of architectural state than the general registers. Some instructions write a destination register and update flags (ADD), some update only flags (CMP), and some do neither. A single combined enable would not be able to express that.

B. MUXes and Enable Signals

On the write side, WriteEnable and the 4-bit WriteAddress feed a 4-to-16 decoder that produces RegisterEnable[0:15], a one-hot set of per-register load-enables, where RegisterEnable[i] = WriteEnable && (WriteAddress == i). WriteData itself is not muxed as it fans out unconditionally to all 16 registers' D-inputs. The only thing that determines whether a given register actually latches the new value on the next clock edge is whether that register's own RegisterEnable[i] bit happens to be high. Every other register's enable stays low and it simply holds its current value. FlagWriteEnable works

as a separate, independent single-bit enable that controls only StoredFlags, decoupled entirely from the register write path, so flags can update alongside a register write, in place of one, or not at all in a given cycle.

On the read side, ReadAddressA and ReadAddressB each drive a 16:1 data mux, implemented through array indexing (Registers[ReadAddressA]), which selects one of the sixteen registers to drive ReadDataA and ReadDataB respectively. This mux exists in duplicate, once per port, so that both operands of an instruction can be read out in the same cycle without the two ports contending for the same hardware. Layered on top of each data mux is a second, 2:1 mux controlled by ReadEnableA or ReadEnableB. When the enable is asserted, the port outputs the value the data mux selected. When it's deasserted, the port outputs 16'b0 instead, overriding whatever the address happened to select.

Put together, on each clock edge the decoder determines which single register, if any, accepts WriteData, while independently and continuously the two read muxes drive two registers' current values out to ReadDataA and ReadDataB, gated to zero whenever their respective enable is low. Because the write is clocked but the reads are combinational, a register being written in the current cycle still reads out its old value during that same cycle. The new value only becomes visible starting the next cycle.

C. Datapath Testing

The register file was tested using a testbench with a 50 MHz clock and a reference model that stored the expected register and flag values. An initial reset was applied to check that all stored values were cleared to zero. Each of the 16 registers was then written with a unique pattern and its bitwise complement. After each write, both read ports were used to compare all register values against the reference model, checking that only the selected register changed. The read ports were also tested with matching and different addresses, and their enables were disabled individually and together to check that disabled outputs returned zero.

Additional tests checked write timing by pulsing WriteEnable between rising clock edges and observing read outputs around a register update. Flag storage was tested by writing different patterns, checking that flags retained their values when FlagWriteEnable was low, and exercising register and flag writes independently. Reset was then asserted between clock edges to check asynchronous clearing, followed by another write to check operation after reset. Throughout the simulation, the testbench counted comparisons and reported any mismatches, ultimately resulting in no errors.

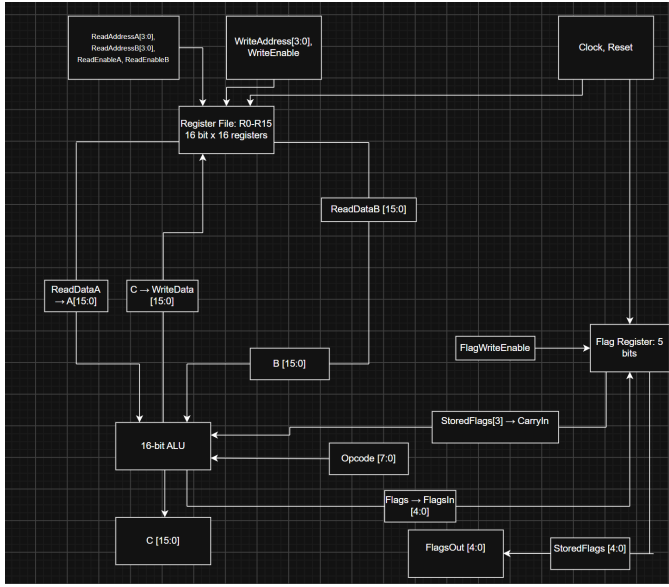


Fig. 1. The 16-bit datapath contains a register file, a register/immediate multiplexer, an ALU, and a five-bit flag register. Read port A/B supplies ALU input A/B, respectively. The ALU result feeds back to the register file, and its flags are stored when FlagWriteEnable is asserted. The stored carry flag supplies CarryIn. All control signals originate outside the diagram. The register-file block includes its write decoder and two read multiplexers. The flag register is drawn separately for clarity. All signals without a specified bus width are one bit.

IV. TESTING METHODOLOGY

In a similar manner to the testing of the register file, the Fibonacci datapath was tested using testbenches with a 50 MHz clock.

The Fibonacci testbench modeled the expected sequence in the fib array, beginning with 1 and 1 and calculating each subsequent entry as the sum of the previous two. The run_and_check_program task checked every execution state from 1 through 17. In each state, it compared StateOut with the expected state, checked the combinational ALUResult during arithmetic operations, and used check_regs to inspect all 16 registers. At state s , the first $s-1$ registers were expected to contain completed results, while the remaining registers were expected to remain zero. This accounts for the fact that each state's result is written on the clock edge that advances to the next state. Checks performed after clock edges included a 1 ns delay to allow sequential updates and combinational outputs to settle.

Three execution scenarios were included: normal execution after startup reset, restarting after halt, and resetting during execution at state 9. The apply_reset task asserted reset on a falling clock edge and checked that the registers cleared before the next rising edge. It then checked that the FSM returned to state 0, Halt cleared, and the stored flags became zero. At the end of each complete run, the testbench checked that R15 and DisplayValue contained 987 (16'h03DB). It continued checking for five additional clock cycles to verify that the FSM remained halted and preserved the register contents. The comparison tasks used case inequality ($!=$) so

```
# run 1000ns
reset to start, then run program
StateEnded if state 17: 17 | Result in register 15: 0 987
reset from halt
xsim: Time (s): cpu = 00:00:06 ; elapsed = 00:00:05 . Memory (MB): peak = 2226.531 ; gain = 0.000
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_fibonacci_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:08 ; elapsed = 00:00:17 . Memory (MB): peak = 2226.531 ; gain = 0.000
run 2 ms
reset mid program
1403 passes, 0 errors
All pass
$finish called at time : 1891 ns : File "C:/VerilogProjects/ALU_datapath_regfile/tb_fibonacci.v" Line 173
```

Fig. 2. Fibonacci testbench simulation log showing startup execution, reset from halt, and reset during execution. The simulation reports a final result of 987 in R15 and completes at 1891 ns with 1,403 checks and zero errors.

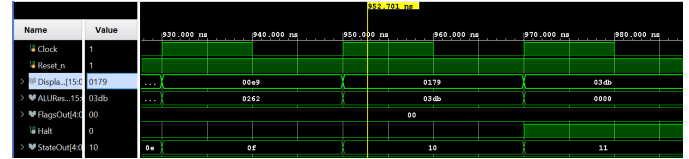


Fig. 3. Simulation waveform showing the final Fibonacci calculations and transition to halt. At 950 ns, StateOut becomes hexadecimal 10 (state 16), and ALUResult is 03DB (987). At 970 ns, the FSM enters hexadecimal 11 (state 17), asserts Halt, and displays 03DB. FlagsOut remains zero, while ALUResult becomes zero because the halt state selects NOP.

unknown or high-impedance values would count as failures, and timeout checks prevented an incorrect controller from running indefinitely.

These tests cover the Fibonacci controller's normal state sequence, immediate and register operand selection, ALU writeback, preservation of previously written registers, reset recovery, and halt behavior. The stored flags were expected to remain zero because all Fibonacci calculations produced positive, nonzero results without carry or signed overflow. Consequently, this test does not establish correctness for nonzero arithmetic flags, other ALU operations, or recovery from an invalid FSM state. Those behaviors require additional directed tests.

In addition figures 2 and 3 were included to support verification of the Fibonacci controller and datapath. Figure 2 documents the successful completion of the automated checks, including reset and restart scenarios. Figure 3 provides timing evidence that the final calculation is followed by the expected halt state on a rising clock edge, with the correct result retained for display.

V. FSM DIAGRAM AND STATE EXPLANATIONS

The FSM uses clk and reset as inputs and changes state on each rising clock edge. If reset is high, any state returns to S_RESET; otherwise, the FSM follows the arrows, with invalid states also returning to S_RESET. Both read enables remain high, and StateOut indicates the current state. In S_RESET, all addresses, RI, Immediate, WriteEnable (shown as WE), Fe (flag register enable), and Halt are zero. During initialization, RI selects immediate 1, WriteEnable and Fe are high, and Rsrc and WriteAddress select R0 or R1 while Rdest remains zero. All Fibonacci calculation states share the same control settings: RI and Immediate are zero, WriteEnable and Fe are high, and Halt is low. Only the addresses change to read the previous two registers and write the next one. In S_HALT,

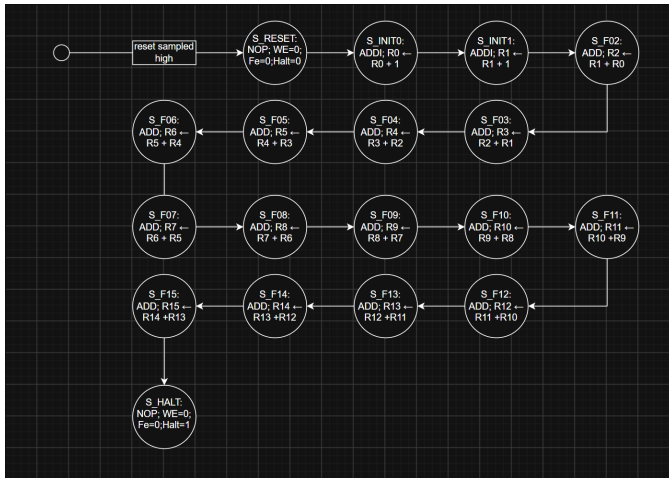


Fig. 4. Fibonacci Moore FSM showing all 18 states and normal state transitions. The controller initializes R0 and R1, calculates each remaining Fibonacci value, and enters a persistent halt state. The global reset rule applies to every state and takes priority over the drawn transitions.

writes are disabled, Halt is high, and Rdest selects R15 for display. The other addresses, RI, and Immediate are zero.

It works by controlling the register file and ALU through a fixed sequence of states. When reset is asserted, the FSM returns to S_RESET on the next rising clock edge, while the register file's reset clears its registers and flags. After reset is released, S_INIT0 and S_INIT1 use add-immediate operations to set R0 and R1 to 1. States S_F02 through S_F15 then select the previous two registers, add their values, and store the result in the next register. Each arithmetic result and its flags are written on the rising clock edge that advances the FSM. After calculating 987 for R15, the controller enters S_HALT, disables register and flag writes, asserts Halt, and selects R15 for display. It remains halted until another reset restarts the sequence.